# Documentação do Projeto

Sistema de Cinema (sist_cinema) · Python + MySQL

Novembro de 2026

**Conteúdo**


1. Especificações técnicas
2. Requisitos funcionais
3. Requisitos não funcionais
4. Regras de negócio
5. Estrutura de banco de dados
6. Manual operacional


## 1. Especificações técnicas

O sistema é um aplicativo desktop em Python que cadastra gêneros, cinemas, salas, filmes e sessões em um banco MySQL, com interface Tkinter nas cores roxo e amarelo.

| Item | Especificação |
|---|---|
| Linguagem | Python 3.10 ou superior (desenvolvido em 3.12) |
| Interface gráfica | Tkinter / ttk, da biblioteca padrão, com o tema `clam` personalizado |
| Banco de dados | MySQL 8 ou superior, engine InnoDB, charset `utf8mb4` |
| Driver de acesso | `mysql-connector-python` 8.0 ou superior |
| Configuração | `python-dotenv` 1.0 ou superior; credenciais no arquivo `.env` |
| Controle de versão | Git, com um commit por funcionalidade |
| Sistema operacional | Windows (usado no desenvolvimento); Linux e macOS com Tkinter instalado |

### Arquitetura em camadas

Cada camada conversa apenas com a de baixo.

1. **Telas** (`telas/`): formulário e lista em Tkinter. A tela `base.py` é um CRUD genérico, e cada `tela_*.py` só declara as colunas e os campos do cadastro.
2. **Repositórios** (`repositorios/`): todo o SQL do sistema. Cada tabela oferece `listar`, `buscar`, `inserir`, `atualizar`, `excluir` e `opcoes`.
3. **Banco** (`database.py` e `config.py`): conexão, transação com commit ou rollback, `consultar`, `executar` e execução de scripts `.sql`.

### Estrutura de pastas

```
cinepy/
├── main.py              ponto de entrada
├── config.py            leitura do .env
├── database.py          conexão, transação, consultar e executar
├── schema.sql           criação do banco e das tabelas
```

```
├── dados_exemplo.sql    dados de teste
├── popular.py           carrega os dados de exemplo
├── requirements.txt     dependências
├── .env.example         modelo das credenciais
├── .env                 credenciais (fora do Git)
├── repositorios/        SQL de cada tabela
│   ├── generos.py
│   ├── cinemas.py
│   ├── salas.py
│   ├── filmes.py
│   └── sessoes.py
└── telas/
    ├── tema.py          cores e estilos
    ├── janela.py        janela principal e abas
    ├── base.py          tela de CRUD genérica
    └── tela_*.py        uma tela por cadastro
```

### Decisões técnicas

- Os valores vão separados do SQL (`%s`), o que evita SQL injection.
- `transacao()` faz commit no fim ou rollback em caso de erro. Gravar um filme junto com seus gêneros usa essa transação: ou grava os dois, ou nenhum.
- A camada de telas nunca escreve SQL. Trocar de banco afeta só `repositorios/` e `database.py`.
- Na tela, datas aparecem como `dd/mm/aaaa` e preços aceitam vírgula; no banco, preço é `DECIMAL(8,2)`.

## 2. Requisitos funcionais

O sistema permite cadastrar, consultar, alterar e excluir os cinco cadastros do cinema, ligando filmes a gêneros e sessões a salas.

| Código | Requisito | Onde |
|---|---|---|
| RF01 | Cadastrar, listar, editar e excluir gêneros (nome obrigatório, descrição opcional). | Gêneros |
| RF02 | Cadastrar, listar, editar e excluir cinemas (nome, endereço, cidade, UF e telefone). A UF é escolhida em uma lista. | Cinemas |
| RF03 | Cadastrar, listar, editar e excluir salas de um cinema, com capacidade e tipo (Convencional, VIP, IMAX, 4DX ou 3D). | Salas |
| RF04 | Cadastrar, listar, editar e excluir filmes (título, duração, classificação, data de lançamento e sinopse). | Filmes |
| RF05 | Associar um ou mais gêneros a cada filme, por seleção múltipla. | Filmes |
| RF06 | Cadastrar, listar, editar e excluir sessões (filme, sala, início, idioma, formato e preço). | Sessões |
| RF07 | Mostrar nas listas o nome do cinema, da sala, do filme e dos gêneros, e não seus números de identificação. | Todas |
| RF08 | Filtrar a lista enquanto o usuário digita na caixa Buscar, comparando o texto com todas as colunas. | Todas |
| RF09 | Preencher o formulário ao selecionar uma linha. O botão Salvar insere se nada estiver selecionado e atualiza se houver um registro selecionado. | Todas |
| RF10 | Pedir confirmação antes de excluir um registro. | Todas |

| Código | Requisito | Onde |
|---|---|---|
| RF11 | Avisar campo obrigatório vazio e valor inválido (número ou data) antes de gravar. | Todas |
| RF12 | Traduzir erros do banco em mensagens claras: item em uso por outro cadastro e registro duplicado. | Todas |
| RF13 | Atualizar as listas de escolha (cinemas, filmes, salas, gêneros) ao trocar de aba. | Todas |
| RF14 | Criar o banco e as tabelas automaticamente na abertura do sistema. | Inicialização |
| RF15 | Carregar dados de exemplo com `popular.py`, e recomeçar do zero com `popular.py --limpar`. | Linha de comando |

# 3. Requisitos não funcionais

O projeto prioriza integridade dos dados, segurança das credenciais e código fácil de estender, aceitando as limitações de um aplicativo desktop de um usuário por vez.

| Código | Categoria | Requisito |
|---|---|---|
| RNF01 | Integridade | O banco garante as regras com chaves primárias, chaves estrangeiras, `UNIQUE`, `NOT NULL` e `ENUM`, e não depende só da interface. |
| RNF02 | Integridade | Operações que gravam em mais de uma tabela (filme e gêneros) rodam em uma transação: tudo grava ou nada grava. |
| RNF03 | Segurança | Usuário e senha do MySQL ficam no `.env`, que não vai para o Git (`.gitignore`). |
| RNF04 | Segurança | Todo valor digitado vai para o banco como parâmetro (`%s`), nunca concatenado ao SQL. |
| RNF05 | Usabilidade | Interface com tema roxo e amarelo, campos obrigatórios marcados com `*` e mensagens de erro em português. |
| RNF06 | Usabilidade | Datas no formato `dd/mm/aaaa` e `dd/mm/aaaa HH:MM`; preço aceita vírgula ou ponto. |
| RNF07 | Manutenibilidade | Acrescentar um cadastro exige criar a tabela, um repositório e uma tela de poucas linhas, sem alterar `telas/base.py`. |
| RNF08 | Manutenibilidade | SQL, regras de tela e conexão ficam em camadas separadas (telas, repositórios e banco). |
| RNF09 | Portabilidade | Roda em Windows, Linux e macOS com Python 3.10 ou superior e MySQL 8 ou superior. |
| RNF10 | Confiabilidade | Falha de conexão na abertura mostra uma caixa de aviso, e não um erro técnico no terminal. |
| RNF11 | Caracteres | Banco e conexão usam `utf8mb4`, então acentos são gravados e lidos corretamente. |

## Limitações conhecidas

- Não há login nem controle de perfis: quem abre o programa altera tudo.
- Cada consulta abre e fecha uma conexão. Para este volume de dados não faz diferença, mas um pool de conexões seria o próximo passo em um sistema maior.
- Os cadastros não têm paginação: todas as linhas são carregadas na lista.

# 4. Regras de negócio

As regras vivem no banco, por chaves e restrições, e a interface apenas as traduz em avisos. A coluna «Onde» diz quem garante cada uma.

## Regras de cadastro

| Código | Regra | Onde |
|---|---|---|
| RN01 | O nome de um gênero é único. | `UNIQUE` em `generos.nome` |
| RN02 | Um filme pode ter vários gêneros e um gênero pode estar em vários filmes. O mesmo par filme e gênero não se repete. | Tabela `filmes_generos`, chave primária composta |
| RN03 | Todo filme precisa de ao menos um gênero. | Tela (seleção obrigatória) |
| RN04 | Todo filme tem título, duração em minutos e classificação (L, 10, 12, 14, 16 ou 18). A classificação padrão é L. | `NOT NULL` e `ENUM` |
| RN05 | Toda sala pertence a exatamente um cinema. | `salas.id_cinema NOT NULL` com chave estrangeira |
| RN06 | O nome da sala é único dentro de um mesmo cinema. Cinemas diferentes podem ter uma «Sala 1» cada. | `UNIQUE (id_cinema, nome)` |
| RN07 | O tipo da sala é Convencional, VIP, IMAX, 4DX ou 3D. | `ENUM` |
| RN08 | Toda sessão tem filme, sala, início, idioma (Dublado ou Legendado), formato (2D ou 3D) e preço. | `NOT NULL`, `ENUM` e chaves estrangeiras |
| RN09 | Uma sala não pode ter duas sessões começando no mesmo horário. | `UNIQUE (id_sala, data_hora_inicio)` |
| RN10 | O cinema de uma sessão é o da sua sala. A sessão não guarda o cinema, para os dois nunca se contradizerem. | Modelagem |

## Regras de exclusão

| Ao excluir | O que acontece | Chave estrangeira |
|---|---|---|
| Cinema | Exclui suas salas e, por elas, as sessões dessas salas. | `salas.id_cinema`: `CASCADE` |
| Sala | Exclui as sessões da sala. | `sessoes.id_sala`: `CASCADE` |
| Filme | Remove as ligações com gêneros. É bloqueado se o filme tiver sessões. | `filmes_generos.id_filme`: `CASCADE`; `sessoes.id_filme`: `RESTRICT` |
| Gênero | É bloqueado se algum filme usar o gênero. | `filmes_generos.id_genero`: `RESTRICT` |
| Sessão | Exclui só a sessão. | Nenhuma tabela depende dela |

O `CASCADE` fica onde o registro filho não faz sentido sem o pai. O `RESTRICT` fica onde apagar perderia histórico, como as sessões de um filme, que podem ter ingressos vendidos.

## O que o schema ainda não garante

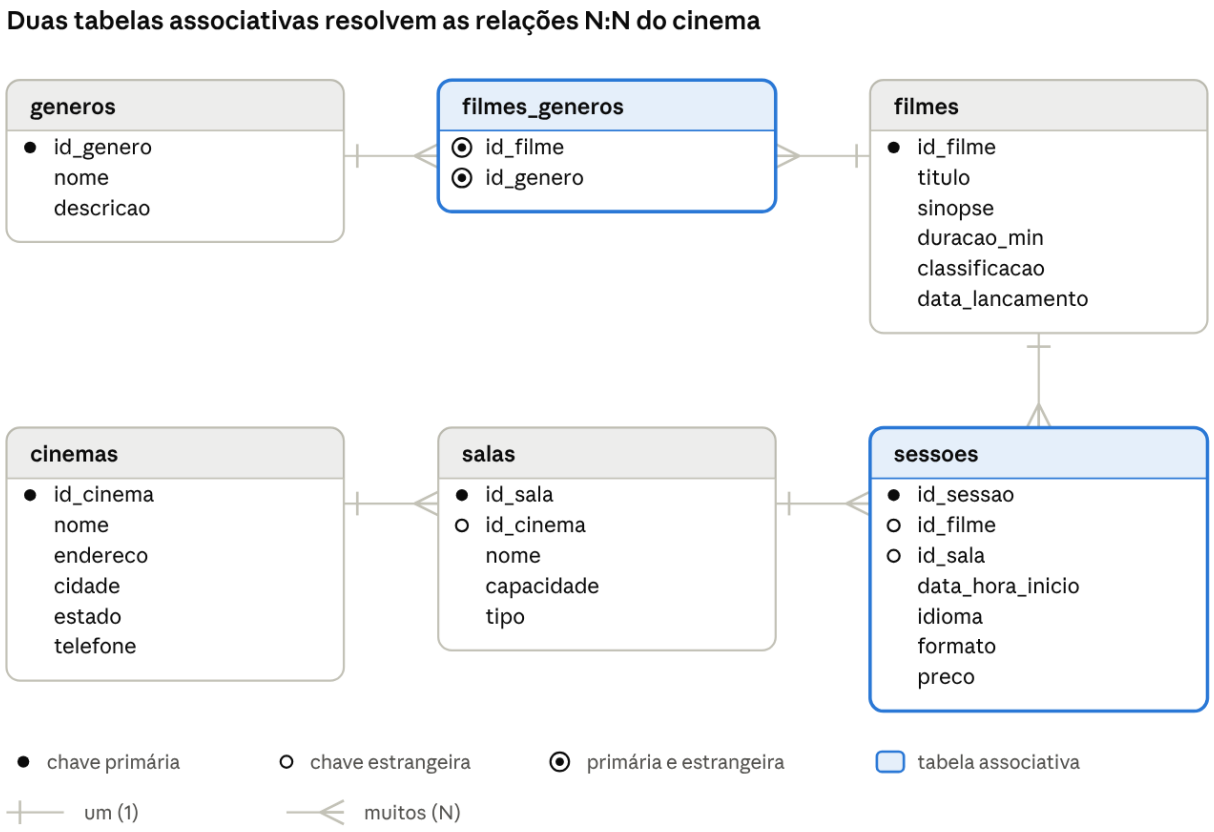
- Duração, capacidade e preço: a tela confere que são números, mas o banco aceita zero ou negativo. Restrições `CHECK` fechariam essa lacuna.
- Sobreposição de sessões: o `UNIQUE` só impede o mesmo horário exato. Um filme de duas horas e outro começando uma hora depois, na mesma sala, são aceitos.

- Filme sem gênero: a tela exige, mas um `INSERT` direto no banco não é barrado.

## 5. Estrutura de banco de dados

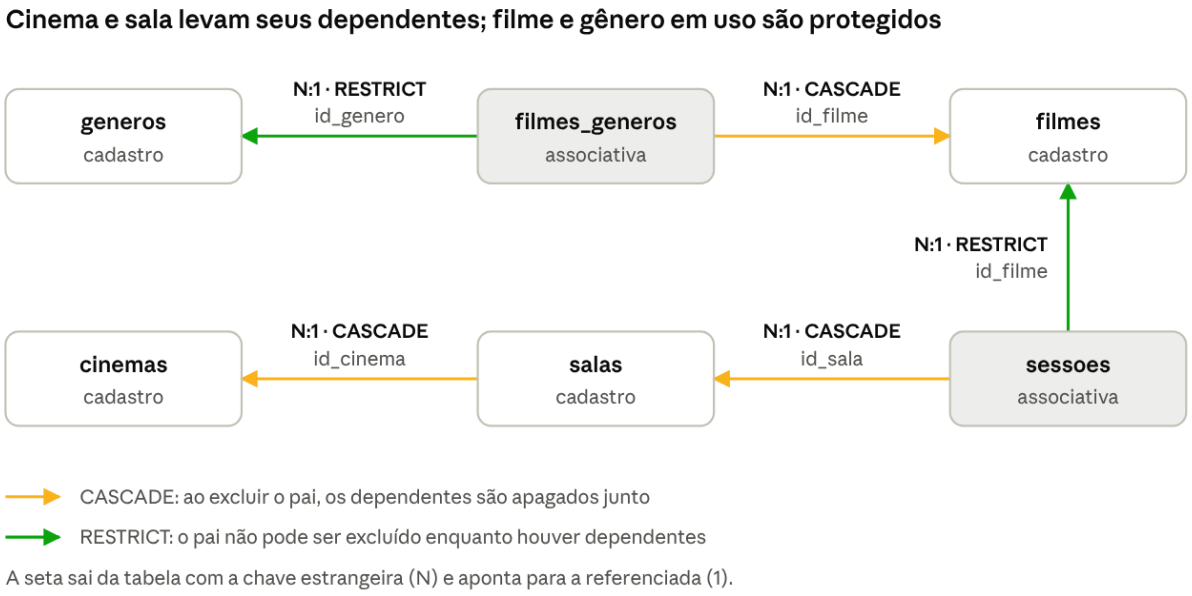
O banco `sist_cinema` tem seis tabelas: quatro cadastros (`generos`, `filmes`, `cinemas` e `salas`), uma tabela associativa N:N (`filmes_generos`) e `sessoes`, que liga um filme a uma sala em um horário. Todas usam InnoDB e `utf8mb4`. O arquivo `schema.sql` é a fonte oficial da estrutura.

### Diagrama entidade-relacionamento




Cada linha liga duas tabelas: a barra marca o lado «um» e o pé de galinha, o lado «muitos». `filmes_generos` e `sessoes`, em destaque, resolvem as relações N:N.

## Relacionamentos




Cinema e sala levam seus dependentes junto (`CASCADE`); filme e gênero em uso ficam protegidos (`RESTRICT`). A tabela `filmes_generos` combina as duas regras: some junto com o filme, mas segura o gênero.

| Entidades | Tipo | Como é resolvido |
|---|---|---|
| `generos` e `filmes` | N:N (um filme tem vários gêneros; um gênero está em vários filmes) | Tabela associativa `filmes_generos`, com chave primária composta |
| `filmes` e `salas` | N:N (um filme passa em várias salas; uma sala exibe vários filmes) | Tabela associativa `sessoes`, com dados próprios: início, idioma, formato e preço |
| `cinemas` e `salas` | 1:N (um cinema tem várias salas; uma sala pertence a um cinema) | Chave estrangeira `salas.id_cinema` |
| `cinemas` e `sessoes` | 1:N, de forma indireta | Pela sala: a sessão não guarda o cinema |

As regras de exclusão de cada chave estrangeira estão na seção 4.

## Dicionário de dados

Cada tabela tem uma chave primária `AUTO_INCREMENT` do tipo `INT UNSIGNED`, exceto `filmes_generos`, que usa chave composta.

### generos

| Coluna | Tipo | Nulo | Chave e regras |
|---|---|---|---|
| `id_genero` | `INT UNSIGNED` | Não | Chave primária |
| `nome` | `VARCHAR(60)` | Não | `UNIQUE` |
| `descricao` | `VARCHAR(255)` | Sim | |

### filmes

| Coluna | Tipo | Nulo | Chave e regras |
|---|---|---|---|
| id_filme | INT UNSIGNED | Não | Chave primária |
| titulo | VARCHAR(150) | Não | |
| sinopse | TEXT | Sim | |
| duracao_min | SMALLINT UNSIGNED | Não | Duração em minutos |
| classificacao | ENUM('L','10','12','14','16','18') | Não | Padrão L |
| data_lancamento | DATE | Sim | |

### filmes_generos

| Coluna | Tipo | Nulo | Chave e regras |
|---|---|---|---|
| id_filme | INT UNSIGNED | Não | Chave primária composta; chave estrangeira para filmes, CASCADE |
| id_genero | INT UNSIGNED | Não | Chave primária composta; chave estrangeira para generos, RESTRICT |

### cinemas

| Coluna | Tipo | Nulo | Chave e regras |
|---|---|---|---|
| id_cinema | INT UNSIGNED | Não | Chave primária |
| nome | VARCHAR(100) | Não | |
| endereco | VARCHAR(200) | Não | |
| cidade | VARCHAR(80) | Não | |
| estado | CHAR(2) | Não | Sigla da UF |
| telefone | VARCHAR(20) | Sim | |

### salas

| Coluna | Tipo | Nulo | Chave e regras |
|---|---|---|---|
| id_sala | INT UNSIGNED | Não | Chave primária |
| id_cinema | INT UNSIGNED | Não | Chave estrangeira para cinemas, CASCADE |
| nome | VARCHAR(50) | Não | UNIQUE (id_cinema, nome) |
| capacidade | SMALLINT UNSIGNED | Não | Lugares da sala |
| tipo | ENUM('Convencional','VIP','IMAX','4DX','3D') | Não | Padrão Convencional |

### sessoes

| Coluna | Tipo | Nulo | Chave e regras |
|---|---|---|---|
| id_sessao | INT UNSIGNED | Não | Chave primária |
| id_filme | INT UNSIGNED | Não | Chave estrangeira para filmes, RESTRICT |

| Coluna | Tipo | Nulo | Chave e regras |
|---|---|---|---|
| id_sala | INT UNSIGNED | Não | Chave estrangeira para salas, CASCADE |
| data_hora_inicio | DATETIME | Não | UNIQUE (id_sala, data_hora_inicio) |
| idioma | ENUM('Dublado','Legendado') | Não | Padrão Dublado |
| formato | ENUM('2D','3D') | Não | Padrão 2D |
| preco | DECIMAL(8,2) | Não | Preço em reais |

# 6. Manual operacional

Para usar o sistema: instale as dependências, informe a senha do MySQL no .env, rode python main.py e cadastre nesta ordem: gêneros, cinemas, salas, filmes e sessões.

## Instalação

É preciso ter Python 3.10 ou superior e o MySQL 8 ou superior em execução.

1. Abra o terminal na pasta do projeto.
2. Opcional: crie e ative um ambiente virtual com python -m venv .venv. No Windows (prompt ou PowerShell) ative com .venv\Scripts\activate; no Git Bash, com source .venv/Scripts/activate; no Linux e macOS, com source .venv/bin/activate.
3. Instale as dependências com python -m pip install -r requirements.txt.

## Configuração do .env

Crie o arquivo .env na raiz do projeto (há um modelo em .env.example) e troque sua_senha pela senha real do MySQL. Não use aspas nem espaços.

```
MYSQL_HOST=localhost
MYSQL_PORT=3306
MYSQL_USER=root
MYSQL_PASSWORD=sua_senha
MYSQL_DATABASE=sist_cinema
```

O .env deve estar no .gitignore para a senha não ir para o Git. Se faltar uma variável, o sistema usa o padrão: localhost, porta 3306, usuário root e senha vazia. Mantenha MYSQL_DATABASE=sist_cinema, porque o schema.sql sempre cria o banco com esse nome.

## Execução

| Comando | O que faz |
|---|---|
| python main.py | Cria o banco e as tabelas se ainda não existirem e abre o sistema. |
| python popular.py | Carrega os dados de exemplo. Só roda se o banco estiver vazio. |
| python popular.py --limpar | Apaga todos os dados e carrega os de exemplo de novo. |

## Usando as telas

Cada aba tem o formulário à esquerda e a lista à direita. As listas de escolha dependem de outros cadastros, então siga a ordem: Gêneros, Cinemas, Salas, Filmes e Sessões.

| Para | Faça |
|---|---|
| Cadastrar | Preencha o formulário, sem nada selecionado na lista, e clique em Salvar. |
| Editar | Clique na linha, altere os campos e clique em Salvar. |
| Excluir | Clique na linha, clique em Excluir e confirme. |
| Começar um novo cadastro | Clique em Novo para limpar o formulário. |
| Procurar | Digite na caixa Buscar; a lista filtra enquanto você digita. |
| Marcar vários gêneros de um filme | Use Ctrl e clique nos gêneros. |
| Informar datas | Use `dd/mm/aaaa`; nas sessões, `dd/mm/aaaa HH:MM`. |
| Informar preço | Digite com vírgula ou ponto, como `25,50`. |

Campos marcados com `*` são obrigatórios.

## Solução de problemas

| Mensagem | Causa | Solução |
|---|---|---|
| `ModuleNotFoundError: No module named 'mysql'` | A biblioteca não está instalada no Python em uso. | Confira se o ambiente virtual está ativo e rode `python -m pip install -r requirements.txt`. |
| `1045 Access denied for user` | Usuário ou senha do `.env` estão errados. | Corrija `MYSQL_USER` e `MYSQL_PASSWORD`, salve o arquivo e rode de novo. |
| `Can't connect to MySQL server` | O MySQL está desligado, ou host e porta estão errados. | Inicie o serviço do MySQL e confira `MYSQL_HOST` e `MYSQL_PORT`. |
| `mysql: command not found` | A pasta do MySQL não está no PATH do sistema. | Use o MySQL Workbench, ou liste as tabelas com `python -c "import database; print(database.consultar('SHOW TABLES'))"`. |
| `Field 'id_genero' doesn't have a default value` | A tabela `filmes` é de uma versão antiga do schema. O `CREATE TABLE IF NOT EXISTS` não altera tabelas que já existem. | Recrie o banco, o que apaga os dados: `python -c "import database; database.executar('DROP DATABASE sist_cinema'); database.criar_banco()"`. |
| Não é possível excluir: este item está sendo usado em outro cadastro | Existem registros que dependem dele (regra `RESTRICT`). | Exclua antes as sessões do filme, ou os filmes que usam o gênero. |
| Já existe um registro com esses dados | O valor repete uma regra `UNIQUE`: nome de gênero, nome de sala no cinema, ou sala no mesmo horário. | Altere o valor repetido. |